CHAPTER 5B: Server-Side Language
PHP Basics (B)

Topics covered:-

- Arrays in PHP
- Accessing elements of an array
- Using loop to access array elements
- Creating multidimensional arrays
- Using functions to manipulate arrays – merging, sorting and etc.

Types of Arrays

- PHP supports **two types** of arrays:-
  - **Indexed arrays**
  - **Associative arrays**

$students

Index	Element Value
[5]	Herman
[4]	Gillian
[3]	Frankie
[2]	Candy
[1]	Beatrice
[0]	Allan

$marks

key	Element Value
Herman	100
Gillian	100
Frankie	97
Candy	72
Beatrice	68
Allan	63

Indexed Array

- These are arrays where each element is referenced by a **numeric index**.
- The first index / key starts from zero (0).
- The second index / key is one (1) and so on…..

Index/Key	Element / Value
0	23.50
1	40.00
2	100.99
3	56.99
4	88.20
5	97.25

- The above array is of size 6 and store a list of product prices of type double / float.

Associative Array

- This type of array is also called a **hash** / **map**.
- Instead of numeric index, associative array is referenced by a **string key**.

Index/Key	Element / Value
jimmy_choo	590.00
louis_vuitton	1200.00
prada	100.99
valentino	475.00
gucci	88.20
chloe	97.00

- The above array is of size 6 and store a list of handbag prices of type double / float.

Declaring Index and Associative Arrays

- Using the PHP built-in **array()** construct or simply **[]**.
- **Example of an Index array construct:-**

```
$product_prices = [23.50, 40.00, 100.99, 56.99];
```

- **Example of an Associative array construct:-**

```
$handbag_prices = ['jimmy_choo'    =>  590.00,
                   'luois_Vuitton' => 1200.00,
                   'prada'         =>  100.99,
                   'valentino'     =>  475.00];
```

OLD way of creating array:
`$arr = array();`

NEW way of creating array:
`$arr = [];`

Note: an array can have a mix of different types of elements, say an array contains string and integer elements.

Accessing and Changing Array Elements

- All the above can be done by using the index / key of an array.
- Given the array declared and initialize this way:

```
$car = []; // an empty array
$car['brand'] = 'Volkswagen';
$car['model'] = 'Polo';
$car['made']  = 'German';
```

- **Example: Retrieving an array element**

```
echo $car['brand']; // display Volkswagen
```

- **Example: Changing the element value**

```
$car['model'] = 'Beetle';
echo $car['model']; // display Beetle
```

- **Example: Add new element at the end of the array**

```
$car['capacity'] = 3000;
```

NOTE:

- In old-school PHP, associative array is commonly used to represent a record. For example, the **$car** array created previously.

- BUT in modern object-oriented programming, a record is typically structured as an object. For example:

```
$car = new stdClass();
$car->brand    = 'Volkswagen';
$car->model    = 'Polo';
$car->made     = 'German';
$car->capacity = 1000;
```

Use **stdClass()** to create a custom object

Properties can be added as need

Output an Entire Array Using print_r() Function

- **print_r()** function can be used to print the keys and values of an array for **testing** purpose:

```
$fruits = [
   'Apple',
   'Banana',
   'Coconut',
];

print_r($fruits);
```

```
Array
(
    [0] => Apple
    [1] => Banana
    [2] => Coconut
)
```

Extract a Range of Element Using array_slice() Function

- **Syntax:**

array_slice(<array to slice>, <start index>, <length>,
<preserve or not to preserve the index>)

- **Example:**

```
$members = ['Alan', 'Bernard', 'Candy', 'Daniel', 'Eric'];
print_r(array_slice($members, 1, 2));
```

- **Resulted Output:-**

```
Array([0]=>Bernard [1]=>Candy)
```

NOTE: The original array left untouched. In order to preserve the original index of Bernard and Candy, use the optional fourth argument, like:-

```
print_r(array_slice($members, 1, 2, true));
Array([1]=>Bernard [2]=>Candy)
```

count() Function –Size of an Array

- **count()** function returns an integer representing the size / length of the array.
- **Example:**

```
$members = ['Alan', 'Bernard', 'Candy', 'Daniel', 'Eric'];
echo count($members); // 5
```

foreach() Loop

- It is used when you want to travers the entire array sequentially from the beginning of the array until to the end of the array.
- **Example:-**

```
$fruits = ['Apple', 'Banana', 'Coconut'];
foreach ($fruits as $v) {
  echo "$v\n";
}
/*
Apple
Banana
Coconut
*/
```

Output

foreach() Loop

- The previous example only been able to access each array element's value.
- If you wish to access value + key, use the example below:-

```
$fruits = ['Apple', 'Banana', 'Coconut'];
foreach ($fruits as $k => $v) {
  echo "$k - $v\n";
}
/*
0 - Apple
1 - Banana
2 - Coconut
*/
```

Output

foreach() Loop

- If you wish to change each element's value in the array in the foreach loop, just pass the value as a **reference** parameter using the symbol **ampersand (&)**.
- **Example:-**

```
$prices = [1.23, 4.56, 7.89];
foreach ($prices as $p) {
  $p += 2;
}
print_r($prices);
/*
Array
(
  [0] => 1.23
  [1] => 4.56
  [2] => 7.89
)
*/
```

No changes to the prices

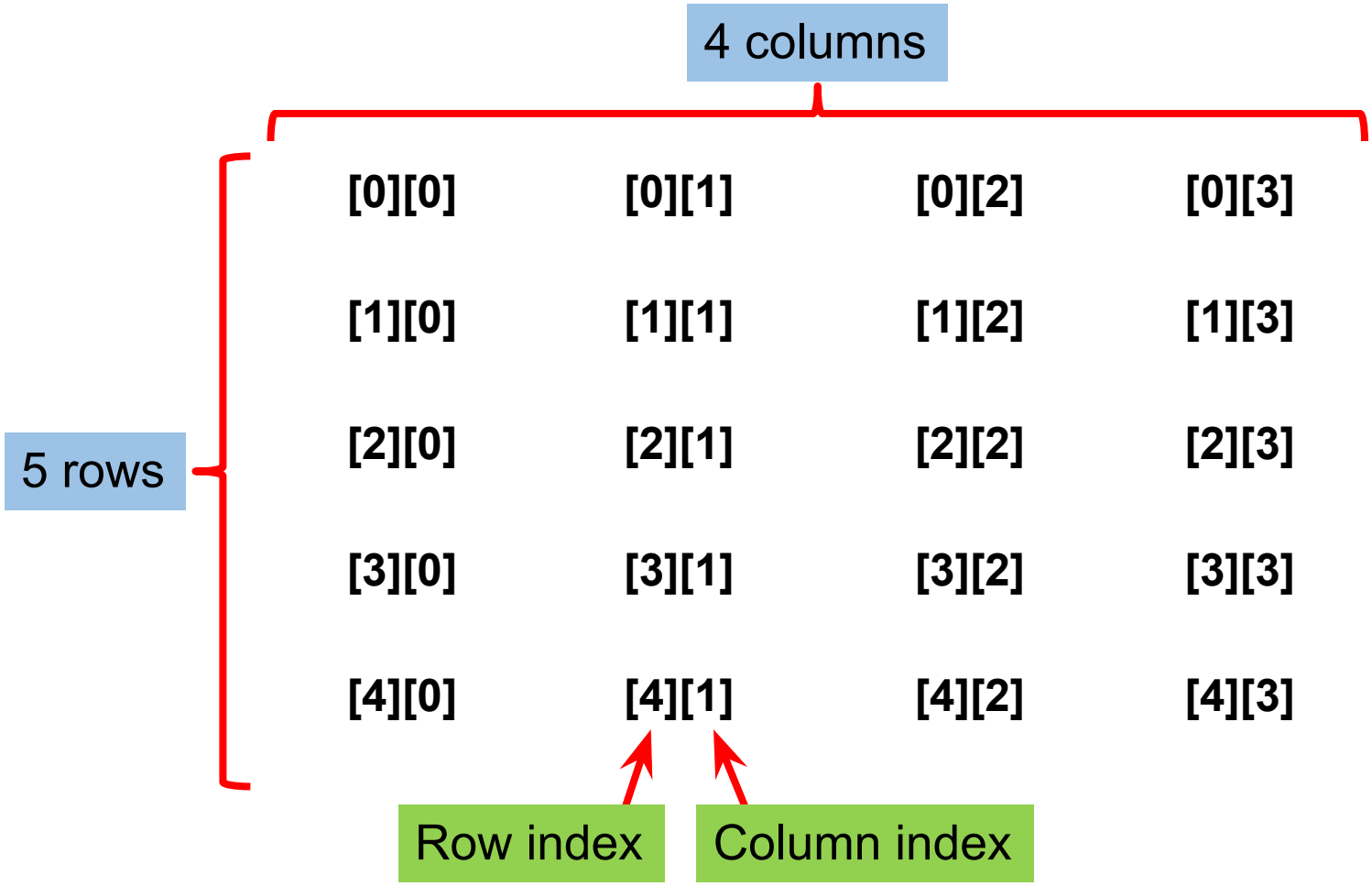
```
$prices = [1.23, 4.56, 7.89];
foreach ($prices as &$p) {
  $p += 2;
}
print_r($prices);
/*
Array
(
  [0] => 3.23
  [1] => 6.56
  [2] => 9.89
)
*/
```

Pass as reference

Prices are updated

Multidimensional Array

- Below shows a two-dimensional array with 5 rows and 4 columns.



```php
// 1st-level
$cars = [
  // 2nd-level
  [
    'brand' => 'Mitsubishi',
    'model' => 'Adventure',
    'made'  => 'Japan',
    'year'  => 1997,
  ],
  [
    'brand' => 'Mini',
    'model' => 'Clubman',
    'made'  => 'British',
    'year'  => 2008,
  ],
  [
    'brand' => 'Kia',
    'model' => 'Sorento',
    'made'  => 'Korea',
    'year'  => 2002,
  ],
];
```

$cars[0]

$cars[1]

$cars[2]

```php
echo '<pre>';
print_r($cars);
echo '</pre>';
```

Resulted Output:-

```
Array
(
    [0] => Array
        (
            [brand] => Mitsubishi
            [model] => Adventure
            [made] => Japan
            [year] => 1997
        )

    [1] => Array
        (
            [brand] => Mini
            [model] => Clubman
            [made] => British
            [year] => 2008
        )

    [2] => Array
        (
            [brand] => Kia
            [model] => Sorento
            [made] => Korea
            [year] => 2002
        )

)
```

PHP Basics (B)

- There are 3 car records stored in the **$cars** array.
- **$cars[0]** retrieves the 1st record as a sub-array.
- To access individual values:

```
echo $cars[0]['brand']; // 'Mitsubishi'
echo $cars[0]['model']; // 'Adventure'
echo $cars[0]['made';  // 'Japan'
echo $cars[0]['year';  // 1997
```

OR

The main array

```
$c = $cars[0];
echo $c['brand']; // 'Mitsubishi'
echo $c['model']; // 'Adventure'
echo $c['made';  // 'Japan'
echo $c['year';  // 1997
```

The sub array

```php
// Object-oriented approach
$cars = [
  (object)[
    'brand' => 'Mitsubishi',
    'model' => 'Adventure',
    'made'  => 'Japan',
    'year'  => 1997,
  ],
  (object)[
    'brand' => 'Mini',
    'model' => 'Clubman',
    'made'  => 'British',
    'year'  => 2008,
  ],
  (object)[
    'brand' => 'Kia',
    'model' => 'Sorento',
    'made'  => 'Korea',
    'year'  => 2002,
  ],
];
```

Cast associative array to object

```php
echo '<pre>';
print_r($cars);
echo '</pre>';
```

Resulted Output:-

```
Array
(
    [0] => stdClass Object
        (
            [brand] => Mitsubishi
            [model] => Adventure
            [made] => Japan
            [year] => 1997
        )

    [1] => stdClass Object
        (
            [brand] => Mini
            [model] => Clubman
            [made] => British
            [year] => 2008
        )

    [2] => stdClass Object
        (
            [brand] => Kia
            [model] => Sorento
            [made] => Korea
            [year] => 2002
        )

)
```

- To access individual values:

```
echo $cars[0]->brand; // 'Mitsubishi'
echo $cars[0]->model; // 'Adventure'
echo $cars[0]->made;  // 'Japan'
echo $cars[0]->year;  // 1997
```

OR

The main array

```
$c = $cars[0];
echo $c->brand; // 'Mitsubishi'
echo $c->model; // 'Adventure'
echo $c->made;  // 'Japan'
echo $c->year;  // 1997
```

The object

Accessing Elements of Multidimensional Array

(1) An array of associative arrays

```
foreach ($cars as $c) {
  echo "$c[brand], $c[model], $c[made], $c[year]\n";
}
```

(2) An array of objects

```
foreach ($cars as $c) {
  echo "$c->brand, $c->model, $c->made, $c->year\n";
}
```

Output

```
Mitsubishi, Adventure, Japan, 1997
Mini, Clubman, British, 2008
Kia, Sorento, Korea, 2002
```

Manipulating Arrays

- This section covers some common and useful array-processing functions.

- These **functions** include:-
  - **Sorting**
  - **Adding and Removing array elements**
  - **Merging arrays together**
  - **Converting between array and string**
  - **Converting array to a list of variables**

- Full reference of array functions: https://www.php.net/manual/en/ref.array.php

Sorting

- Commonly used PHP array sorting functions:
  - **sort()** and **rsort()** – For sorting indexed array by values.
  - **asort()** and **arsort()** – For sorting associative array by values.
  - **ksort()** and **krsort()** – For sorting associative array by keys.

NOTE: The **r** version of the function sort an array in **reversed (descending)** order

- Sort array with **user-defined comparison function**:
  - **usort()** – Sort indexed array by values.
  - **uasort()** – Sort associative array by values.
  - **uksort()** – Sort associative array by keys.

NOTE: These functions are useful for sorting a multidimensional array or an array of objects

sort() and rsort() – Indexed Array

```
$fruits = ['Coconut', 'Apple', 'Banana'];

sort($fruits);
print_r($fruits);
// Array([0]=>Apple [1]=>Banana [2]=>Coconut)

rsort($fruits);
print_r($fruits);
// Array([0]=>Coconut [1]=>Banana [2]=>Apple)
```

Sort by values. **NOTE:** The index is not preserved

asort() and arsort() – Associative Array

```
$prices = [
  'B' => 7.89,
  'C' => 1.23,
  'A' => 4.56,
];

asort($prices);
print_r($prices);
// Array([C]=>1.23 [A]=>4.56 [B]=>7.89)

arsort($prices);
print_r($prices);
// Array([B]=>7.89 [A]=>4.56 [C]=>1.23)
```

Sort by values. **NOTE:** The key is preserved

ksort() and krsort() – Associative Array - key

```
$prices = [
  'B' => 7.89,
  'C' => 1.23,
  'A' => 4.56,
];

ksort($prices);
print_r($prices);
// Array([A]=>4.56 [B]=>7.89 [C]=>1.23)

krsort($prices);
print_r($prices);
// Array([C]=>1.23 [B]=>7.89 [A]=>4.56)
```

Sort by keys

usort() – multidimensional array

```
$fruits = [
  ['name' => 'Apple'  , 'price' => 7.89],
  ['name' => 'Banana' , 'price' => 1.23],
  ['name' => 'Coconut', 'price' => 4.56],
];

usort($fruits, fn($a, $b) => $a['price'] <=> $b['price']);
print_r($fruits);
```

Use the **spaceship operator:**
$a < $b return -1 (negative)
$a = $b return 0
$a > $b return +1 (positive)

User-defined compare function: Tells PHP how to compare 2 array items **$a** and **$b**. PHP will use the result returned by this function to sort the array items.

```
Array
(
  [0]=>Array([name]=>Banana  [price]=>1.23)
  [1]=>Array([name]=>Coconut [price]=>4.56)
  [2]=>Array([name]=>Apple   [price]=>7.89)
)
```

usort() – array of objects

```
$fruits = [
  (object)['name' => 'Apple'  , 'price' => 7.89],
  (object)['name' => 'Banana' , 'price' => 1.23],
  (object)['name' => 'Coconut', 'price' => 4.56],
];

usort($fruits, fn($a, $b) => $a->price <=> $b->price);
print_r($fruits);
```

```
Array
(
  [0]=>stdClass Object([name]=>Banana  [price]=>1.23)
  [1]=>stdClass Object([name]=>Coconut [price]=>4.56)
  [2]=>stdClass Object([name]=>Apple   [price]=>7.89)
)
```

NOTE: To sort items in descending order, can call **array_reverse($fruits)** after the **usort()** function

Adding and Removing Array Elements

- To push a new item into an array:

```
$arr = [];     // []
$arr[] = 123; // [123]
$arr[] = 456; // [123, 456]
$arr[] = 789; // [123, 456, 789]
```

Simply assign new value to the array with **empty square brackets** (without specifying and index)

- In addition, PHP support for another 5 powerful functions to add and remove array elements:-

Functions	Descriptions
array_unshift()	Add elements at the start of the array
array_shift()	Removes the first element from the start of the array
array_push()	Add elements at the end of the array
array_pop()	Removes the last element from the end of the array
array_splice()	Removes / Add elements at any point in the array

array_unshift() and array_shift() functions

- **array_unshift()** – Prepend one or more elements to the **beginning** of an array. Return the new elements count.
- **array_shift()** – Shift an element off the **beginning** of an array. Return the shifted value, or null if the array is empty.

```
$arr = ['X', 'D'];                          // ['X','D']
echo array_shift($arr);                     // 'X'  ['D']
echo array_unshift($arr, 'C');              //  2   ['C','D']
echo array_unshift($arr, 'A', 'B');         //  4   ['A','B','C','D']
```

- **<u>NOTE:</u>**
  - Numeric index ⇒ modified to start counting from zero.
  - String key ⇒ won't be changed.

array_push() and array_pop() functions

- **array_push()** – Append one or more elements to the **end** of an array. Return the new elements count.
- **array_pop()** – Pop an element off the **end** of an array. Return the popped value, or null if the array is empty.

```
$arr = ['A', 'X'];                          ['A','X']
echo array_pop($arr);              // 'X'   ['A']
echo array_push($arr, 'B');        //  2    ['A','B']
echo array_push($arr, 'C', 'D');   //  4    ['A','B','C','D']
```

- **NOTE:**
  - **array_push($arr, 'B')** has same effect as **$arr[] = 'B'**.
  - If only adding 1 element, use **$arr[] = 'B'** approach, as there is no overhead of calling a function.

array_splice() function

- **array_splice()** – Remove a portion of an array and replace it with something else. Return an array contains the removed items.

Optional **replacement items** (as an array) to be inserted after removal

```
$arr = ['A', 'X', 'Y', 'D'];
array_splice($arr, 1, 2, ['B', 'C']); // ['A','B','C','D']
```

Offset or **index** the operation takes place

Number of items to remove.
0 = Do not remove any item.
null = Remove until the end of array (default)

```
// From index 1, remove 2 items, without replacement
array_splice($arr, 1, 2);
// From index 1, remove all items, without replacement
array_splice($arr, 1, null); // OR
array_splice($arr, 1);
```

Merging Arrays Together

- **array_merge()** – Merge 2 or more arrays together.
  - Values of later array are appended to the end of previous array.
  - **Numeric index** ⇒ will be renumbered starting from 0.
  - **String key** ⇒ values of later array will overwrite values of previous array if the string keys are identical.

Previous Later

```
$result = array_merge($arr1, $arr2);
```

- **Append** to the end
- **Overwrite** if same string key

Merging Arrays Together

- Merging 2 arrays with numeric index ⇒ indexed arrays:

```
$a = ['A', 'B', 'X'];
$b = ['C', 'D', 'X'];
$c = array_merge($a, $b);
print_r($c);
// Array([0]=>A [1]=>B [2]=>X [3]=>C [4]=>D [5]=>X)
```

- Merging 2 arrays with string key ⇒ associative arrays:

```
$a = ['A'=>1, 'B'=>2, 'X'=>3];
$b = ['C'=>4, 'D'=>5, 'X'=>6];
$c = array_merge($a, $b);
print_r($c);
// Array([A]=>1 [B]=>2 [X]=>6 [C]=>4 [D]=>5)
```

Later value overwrites previous value

Merging Arrays Together

- Modern PHP supports the **array spread operator (...)**. Same effect as **array_merge()** but without the overhead of calling function:

```
$a = ['A', 'B', 'X'];
$b = ['C', 'D', 'X'];
$c = [...$a, ...$b];
print_r($c);
// Array([0]=>A [1]=>B [2]=>X [3]=>C [4]=>D [5]=>X)
```

```
$a = ['A'=>1, 'B'=>2, 'X'=>3];
$b = ['C'=>4, 'D'=>5, 'X'=>6];
$c = [...$a, ...$b];
print_r($c);
// Array([A]=>1 [B]=>2 [X]=>6 [C]=>4 [D]=>5)
```

Merging Arrays Together

- The **array spread operator (...)** spreads the array elements in place, it can be put at the start, middle or end of the new array:

```
$a = ['A', 'B'];
$b = ['C', 'D'];
$c = ['X', ...$a, 'Y', ...$b, 'Z'];
print_r($c);
// Array([0]=>X [1]=>A [2]=>B [3]=>Y [4]=>C [5]=>D [6]=>Z)
```

array_combine function

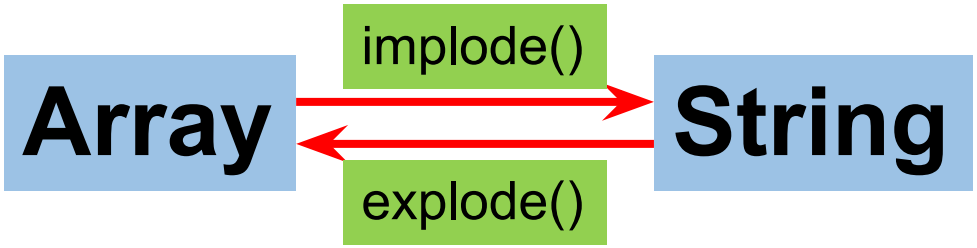
- **array_combine()** – Create a new array by using one array as keys and another array as values.

```
$keys   = ['APP'  , 'BNN'   , 'CCN'    ];
$values = ['Apple', 'Banana', 'Coconut'];
$arr = array_combine($keys, $values);
print_r($arr);
// Array([APP]=>Apple [BNN]=>Banana [CCN]=>Coconut)
```

- **NOTE:** The number of elements in arrays **$keys** and **$values** must be the same.

Converting between array and string

- **Implode()** – Join array elements into a string.
- **explode()** – Split a string into an array.




```
$arr = ['Apple', 'Banana', 'Coconut'];
$s = implode(',', $arr);
print_r($s);
// Apple,Banana,Coconut
```

```
$s = 'Apple,Banana,Coconut';
$arr = explode(',', $s);
print_r($arr);
// Array([0]=>Apple [1]=>Banana [2]=>Coconut)
```

Use **comma (,)** as the separator

Converting Array to a List of Variables

- Conventional way to assign each array elements into variables:

```
$arr = ['A', 'B', 'C'];
$a = $arr[0]; // 'A'
$b = $arr[1]; // 'B'
$c = $arr[2]; // 'C'
```

- This can be cut short the with the **list()** construct:

```
$arr = ['A', 'B', 'C'];
list($a, $b, $c) = $arr;
echo $a, $b, $c; // ABC
```

[0] [1] [2]

This works with indexed array only. It unpacks elements in the sequence of the index. Starting from 0, 1, 2, …

It is not necessary to unpack all elements. Elements can be skipped as follows:

```
$arr = ['A', 'B', 'C'];
list($a, $b    ) = $arr;
list(  , $b, $c) = $arr;
list($a,   , $c) = $arr;
```

Converting Array to a List of Variables

- To unpack **associative array**, provide the keys as follows:

```
$arr = ['A'=>1, 'B'=>2];
list('A'=>$a, 'B'=>$b) = $arr;
echo $a, $b; // 12
```

Values assigned to variables based on matching keys

- Sequence is not important in this case:

```
$arr = ['A'=>1, 'B'=>2];
list('B'=>$b, 'A'=>$a) = $arr;
echo $a, $b; // 12
```

- Similarly, values can be skipped – just don't list the keys:

```
$arr = ['A'=>1, 'B'=>2];
list('B'=>$b) = $arr;
echo $b; // 2
```

Converting Array to a List of Variables

- In **modern PHP**, array can be unpacked using the following syntax:

```
$arr = [1, 2, 3];
[$a, $b] = $arr;
echo $a, $b; // 12
```

This works the same way as the **list()** construct. Values can be skipped if necessary

```
$arr = ['A'=>1, 'B'=>2, 'C'=>3];
['A'=>$a, 'B'=>$b] = $arr;
echo $a, $b; // 12
```

extract() and compact()

- **extract()** = Extract elements of an **associative array** into variables.
- **compact()** = Compact variables into an **associative array**.

```
$arr = [
  'id'   => 1,
  'name' => 'Lisa',
  'cgpa' => 3.8,
];

extract($arr);

echo "$id $name $cgpa";
// 1 Lisa 3.8
```

3 variables are created based on the keys and assigned with the respective values:

- **$id = 1**
- **$name = 'Lisa'**
- **$cgpa = 3.8**

extract() and compact()

- **compact()** does the reverse.

Create an associate array using the variables **$id**, **$name** and **$cgpa**.

The variable names must be written as strings

```
$id   = 1;
$name = 'Lisa';
$cgpa = 3.8;

$arr = compact('id', 'name', 'cgpa');

print_r($arr);
// Array([id]=>1 [name]=>Lisa [cgpa]=>3.8)
```

CHAPTER 5B: Server-Side Language
PHP Basics (B)

Topics covered:-

- **Functions**
- **PHP type hints**
- **Anonymous and arrow functions**
- **Functions and global variables**
- **Pass by reference**

Functions

NOTE: PHP function name is case-insensitive

- Simple PHP function:

```
function sum($a, $b) {
  return $a + $b;
}

echo sum(1, 2); // 3
```

The **return** statement is optional

```
echo SUM(1, 2); // 3
echo SuM(1, 2); // 3
```

- Function with default parameters:

```
function sum($a, $b, $c = 0, $d = 0) {
  return $a + $b + $c + $d;
}

echo sum(1, 2, 3); // 6
```

$c and **$d** are given default values they can be skipped in function call

$c = 3, **$d** = 0

Functions

- Function with variable-length parameter:

```
function sum(...$arr) {
  return array_sum($arr);
}

echo sum();          // 0
echo sum(1);         // 1
echo sum(1, 2);      // 3
echo sum(1, 2, 3);   // 6
```

Use … operator to indicate a variable-length parameter □ values will be stored as array

Able to call the function with different number of parameters

- **NOTE:** The **array_sum()** function returns the sum of all array elements □ values must be integers or floats.

Functions

- Unpack (or destructure) array in function calls:

```
function sum($a, $b) {
  return $a + $b;
}

echo sum(...[1, 2]);    // 3
echo sum(...[1, 2, 3]); // 3
```

Use … operator to unpack array in function calls array values assigned to individual parameters, extra ones will be omitted

NOTE:

- Modern PHP supports type hints:

```
function sum(int $a, int $b): int {
  return $a + $b;
}

echo sum(1, 2);      // 3
echo sum(1.9, 2.9); // 3
echo sum('A', 'B'); // ERROR
```

Supported type hints:
int, **float**, **string**, **bool**, **array**, **object**, **resource**, **callable**, **void**, **null**, **mixed**

Return type

Values are casted to **int**
□ 1 + 2 = 3

- The **mixed** type is the union of all types, except **void** □ means it can be any data type.

NOTE:

- Type hints can be union:

```
function sum(int|float $a, int|float $b): int|float {
  return $a + $b;
}

echo sum(1, 2);      // 3
echo sum(1.9, 2.9); // 4.8
echo sum('A', 'B'); // ERROR
```

```
function sum(?int $a, ?int $b): int {
  return $a + $b;
}

echo sum(null, null); // 0 + 0 = 0
echo sum(1, null);    // 1 + 0 = 0
echo sum(null, 2);    // 0 + 2 = 0
```

Use **?** symbol to indicate a parameter or return value is **nullable**

Anonymous Functions

Function variable

```
$sum = function ($a, $b) {
  return $a + $b;
};

echo $sum(1, 2); // 3
```

Anonymous function □ function without a name

- Useful for one-time function. For example:

```
$a = [1, 2, 3];
$b = array_map(function ($v) {
        return $v * 2;
      }, $a);
// $b = [2, 4, 6]
```

The **array_map** function calls the given function against each array element, and return the resulting new array

Arrow Functions

- **Arrow function =** Shortcut for anonymous function that consists of only **ONE (1)** return statement:

```
$sum = fn($a, $b) => $a + $b;
echo $sum(1, 2); // 3
```

```
$a = [1, 2, 3];
$b = array_map(fn($v) => $v * 2, $a);
// $b = [2, 4, 6]
```

Functions and Global Variables

- In PHP, functions have NO ACCESS to global variables by default.

```
$count = 0;

function increase() {
  $count++; // ERROR
}
```

Global variable

All global variables are accessible via the **$GLOBALS** array, which variable names as array keys

- Use **global** keyword or **$GLOBALS** array to access to global variables:

```
$count = 0;

function increase() {
  global $count;
  $count++; // OK
}
```

OR

```
$count = 0;

function increase() {
  $count = $GLOBALS['count'];
  $count++; // OK
}
```

Pass by Reference

- By default, scalar values and arrays are **passed by value**. Modifying parameters within functions DOES NOT affect the original variables.

```
function foo($n) {
  $n++;
}

$a = 0;
foo($a);
// Result: $a = 0
```

```
function moo($arr) {
  $arr[0] = 10;
}

$b = [1, 2, 3];
moo($b);
// Result: $b = [0, 1, 2]
```

Pass by Reference

- Prefix parameters with **&** symbol to **pass by reference**.
- Modifying them within functions will affect the original variables.

```
function foo(&$n) {
  $n++;
}

$a = 0;
foo($a);
// Result: $a = 1
```

```
function moo(&$arr) {
  $arr[0] = 10;
}

$b = [1, 2, 3];
moo($b);
// Result: $b = [10, 1, 2]
```

Pass by Reference

- **NOTE: Objects** are **passed by reference** by default.

```
function boo($f) {
  $f->name = 'Banana';
}

$f = new stdClass();
$f->id   = 123;
$f->name = 'Apple';
boo($f);
// Result: $f->id   = 123
//         $f->name = 'Banana'
```